scientific reports

Check for updates

OPEN

Adaptive motion planning for legged robots in unstructured terrain using deep reinforcement learning

Mohammad Salah uddin

Legged robots are ideal for navigating unstructured terrain. Unlike wheeled robot platforms, legged robots maintain mobility over rocks, slopes, and uneven surfaces. However, motion planning in such environments remains challenging. The robot must handle unstable ground, limited sensing, and complex body dynamics. Classical control methods often fail to adapt when terrain and friction change unexpectedly. This paper presents an adaptive locomotion framework for quadrupedal robots. The controller is trained entirely in simulation using deep reinforcement learning (DRL). The DRL is trained using proximal policy optimization (PPO) method. During training, the robot observes joint states, body attitude, and local terrain height information. Training follows a curriculum mechanism. The robot starts on flat terrain. The difficulty gradually increases with slopes, height noise, rough surfaces, and friction variation. The reward function encourages forward progress, stability, low foot slip, and energy-efficient motion. Training occurs in Webots with wide domain randomization ranges. Policies learn to select footholds, regulate body attitude, and minimize slip without explicit terrain labels or gait scripts. Several behaviors emerge through learning. These include wider stance on slopes, increased step height on rocks, and hip abduction for balance recovery. To ensure robustness, we apply a multi-step validation process. These include cross-simulator evaluation, domain randomization Monte Carlo tests, physics consistency tests and adversarial disturbances. The controller achieves 94.6% success in Webots and 91.2% in PyBullet without retraining. It recovers from external pushes within 1.6 seconds. Results confirm that the proposed method learns stable, energy-efficient walking with minimal foot slippage across friction coefficients $\mu = 0.1 to 1.0$. This capability supports deployment in search-and-rescue, off-road mobility, and exploration tasks in unknown terrain.

Keywords Legged robots, Motion planning, Deep reinforcement learning, Terrain adaptation, Curriculum learning, Webots simulation, PPO, Quadruped locomotion

Legged robots are nowadays being considered as an important solution for moving in complex and unstructured terrains. From disaster affected areas to very remote extraterrestrial surface, the requirement of autonomous mobility in environment which are not suitable for wheeled or tracked vehicles is increasing day by day. As compared to wheeled robots, legged robots, specially quadruped robots, are having more adaptability in environments having irregular geometry, debris, slopes and gaps. This inspiration taken from biological animals, mainly mammals like dogs and mountain goats, is not only giving better terrain coverage but also providing improved maneuverability, stability and access to areas which were not reachable earlier.

Despite these advantages, motion planning for legged robots remains one of the most challenging problem in the field of robotics. The main difficulty lies in the dynamic interaction between the mechanical structure of the robot and the unpredictable terrain which it has to traverse. Such environments are often presenting discontinuous and unstable surfaces, due to which the robot is required to continuously adjust its center of mass, joint angles and foot placement in real time. The traditional motion planning approaches, which are mostly based on classical control theory or model predictive control (MPC), are ill-suited for these fast changing situations. These methods rely heavily on precise environment models and deterministic behaviors, which falter in the face of incomplete sensing, dynamic ground conditions, and unforeseen disturbances.

Computer Science and Engineering Department, East West University, Aftabnagar, Dhaka 1212, Badda, Bangladesh. email: uddin@ewubd.edu; akash.bangla@gmail.com

In response to these limitations, the researchers are increasingly turning towards learning based methods, specially deep reinforcement learning (DRL), as a viable alternative for generating adaptive locomotion policies. DRL enables the robots to learn control strategies directly by interacting with the environment, without having requirement of explicit modeling of terrain dynamics. Among different DRL algorithms, Proximal Policy Optimization (PPO) have gained prominence due to its robustness, sample efficiency and ability to handle continuous control tasks with high-dimensional state and action spaces. However, direct training of such complex behaviors in unstructured terrains often results in unstable policy or poor generalization performance. At this stage, curriculum learning becomes very much critical.

Inspired from the way how humans and animals are acquiring complex skills, curriculum learning is structuring the learning process by first starting with simple tasks and then gradually increasing the difficulty level. In the context of locomotion, this means training begins on flat, predictable terrain and gradually introduces complexity such as slopes, friction variation, and rough topography. This step by step learning process is not only helping to stabilize the training but also encouraging the eemergence of generalizable locomotion strategies. By the time the agent is exposed to complex terrain, it has already mastered the fundamental principles of gait stability and energy-efficient movement.

In this paper, we present a novel adaptive motion planning framework for quadrupedal robots that uses DRL with curriculum learning to develop robust and efficient locomotor behavior. The proposed system is trained using a custom simulation environment in Webots. The simulation allows systematic generation of different terrain types. These include variations in friction coefficients, surface slopes, and rocky layers. The robot sensory inputs includes both proprioceptive data and terrain related feedback. Proprioceptive information consists of joint position, joint velocity, and applied torque. Terrain feedback is obtained from simulated exteroceptive sensors, which allows the robot to perceive ground conditions and respond to them in real time during locomotion.

Our approach emphasises minimal reliance on pre programmed movements. It also avoids manually tuned control heuristics. Instead of this, the learning process is mostly autonomous and guided by a carefully crafted reward function. The reward function encourages stability. It supports smoother gait transitions. It also promotes forward progression and lower energy usage during motion. Notably, the robot learns adaptive foot placement over time. It compensates for uneven terrain conditions. At the same time, it maintains overall balance and motion efficiency during locomotion tasks.

We evaluate our framework using both quantitative metrics and qualitative analysis. Results from simulated trials demonstrate the effectiveness of the learned policies in navigating challenging terrains, including sloped, low-friction, and rocky environments. The system shows resilience to sensor noise and continues to exhibit stable behavior even when confronted with terrain configurations not encountered during training. Additionally, an ablation study highlights the contribution of each component–DRL, curriculum structure, and reward shaping–to the overall performance of the system. In summary, our contributions are as follows:

- We introduce a DRL-based motion planning system for quadrupedal (four legged) robots that learns stable locomotion in unstructured terrain through curriculum learning.
- We develop a versatile simulation environment with procedurally generated terrains and integrated sensor noise modeling to simulate real-world challenges.
- We design a multi-component reward function that facilitates the emergence of energy-efficient and adaptive gaits.
- We demonstrate that the learned policies generalize well across different terrains and maintain robust performance under sensor uncertainty.
- We validate robust cross-simulator generalization by transferring learned control policies from Webots to PyBullet without re-tuning/re-taining. Webots and PyBullet, both are simulation platform.
- A rigorous five-stage validation process including cross-simulator tests, domain-randomization Monte-Carlo runs, physics-consistency checks, and adversarial-disturbance testing.

The findings of this research have significant implications for real-world applications such as search and rescue, planetary exploration, and off-road logistics. By enabling robots to dynamically adapt to unpredictable ground conditions, this research brings us closer to deploying fully autonomous walking/legged robots capable of operating in environments where human access is limited, unsafe, or risky.

After the introduction, Section "Related work" reviews the related work and positions this study in the broader research context. Section "System architecture" describes the system architecture, outlining each components in details. Section "Learning framework" presents the learning framework and explains the training strategy. Section "Experimental setup" details the experimental setup with implementation steps. Section "Results" reports the primary results. Section "Discussion" discusses the findings and interprets their significance. Section "Validation methodology" introduces the validation methodology used to evaluate the system. Section "Validation results" presents the validation results. Section "Conclusion" concludes the paper and highlights future research directions. The appendix is provided as supplementary material for interested readers.

Related work

The development of autonomous locomotion strategies for legged robots has remained as a long standing objective in the field of robotics. Over the years, researchers have explored different types of control approaches, ranging from rule based motion planning methods to more recent learning based techniques. This section is reviewing the related prior works in important areas such as classical planning, model based control, reinforcement learning, curriculum learning and terrain simulation. It also highlights the limitations present in

existing approaches and explains how the proposed method is contributing towards advancement of the state of the art.

Classical motion planning and model-based control

Traditional approaches for robot locomotion are mostly depending on deterministic control principles which are derived from physical models of the robot and its environment. Algorithms such as A* (star) and Rapidly Exploring Random Trees (RRT) are adapted for foot based locomotion by calculating possible foot positions and body trajectories. These algorithms are working well in known or structured terrain situations. However, they are often failing in dynamic and uncertain environments. This is mainly due to their reliance on precise terrain maps and also because of high computational complexity[1,2].

Model based controllers, including Zero Moment Point (ZMP) and Model Predictive Control (MPC), are providing dynamic stability by continuously optimizing the pose of the robot and its center of mass. These control methods are widely used in bipedal and quadrupedal robotic platforms such as ASIMO and MIT Cheetah. However, such approaches require very accurate dynamic models of the robot system. They are also highly sensitive to modeling errors, sensor noise and unexpected external disturbances, which makes them more brittle and less reliable in real world scenarios[3,4].

Deep reinforcement learning for locomotion

Deep reinforcement learning (DRL) allows robotic agents to learn locomotion and control policies through trial and error interaction with the environment. In contrast to traditional model based approaches, DRL does not require explicitly defined physical models of the robot or surroundings. Due to this advantage, DRL has achieved major success in simulation environments, where agents are able to learn complex behaviors such as walking, running and obstacle avoidance[5,6].

DRL has been widely applied in OpenAI Gym environments such as HalfCheetah, Ant and Walker2D, and more recently in MuJoCo based locomotion benchmarks developed by Google DeepMind. These works demonstrate that DRL agents are capable of learning complex and human like gait patterns. However, many early DRL based approaches were limited mainly to flat simulated terrains and showed poor generalization in real world scenarios. This is largely due to sensor uncertainty and environmental variability[7,8].

In real world robotics applications, deep reinforcement learning (DRL) has been applied to legged robotic platforms such as ANYmal and Spot (from Boston Dynamics), allowing them to learn locomotion behaviors like walking and stair climbing. However, in most practical deployments, hybrid control approaches are commonly used, where DRL is combined with model based controllers or additional safety mechanisms in order to ensure stability and robustness of the system. These robotic systems are usually restricted to controlled operating environments,since achieving generalization across unstructured terrain remains a major challenge due to variable ground conditions and sensor noise[1].

Curriculum learning in robotics

Curriculum learning is inspired from cognitive science and it suggests that learning becomes more effective when tasks are presented in a gradual manner, from easy to more difficult levels. In robotics domain, this principle has been successfully applied to stabilize deep reinforcement learning (DRL) training process, improve convergence, and enhance generalization across different environments[9].

Several recent studies have applied curriculum learning techniques to enhance locomotion performance in robotic system. For instance[10], showed improved control policies by gradually increasing the terrain complexity during the training process. Various terrain interpolation strategies, starting from flat surface and moving towards more rugged terrain, together with task decomposition methods such as learning to stand before walking, have been shown to stabilize the training process and promote better policy generalization. In a similar way[11], proposed a multi-stage curriculum learning approach, where the learning agents were progressively exposed to different terrain conditions and external disturbances, which resulted in significant improvement in bipedal walking performance.

Despite its benefits, curriculum learning is still not commonly adopted as a standard component in deep reinforcement learning (DRL) based locomotion frameworks. In this work, we advance the existing research by integrating a finely staged terrain curriculum directly into the DRL training pipeline. This allows the learning agent to first master basic locomotion skills before moving towards more complex terrains involving friction variation, sensor noise and instability. Such a training strategy helps to address key limitations related to DRL generalization in unstructured environments[12].

Reward design for locomotion tasks

The performance of deep reinforcement learning (DRL) in locomotion tasks depends heavily on the design of the reward function. Common reward components include balance maintenance, energy minimization, target velocity tracking and penalization of undesired behaviors such as foot slippage. However, reward shaping is mostly empirical in nature, and poorly designed reward functions can result in unstable or sub optimal behaviors. Recent research highlights the need for adaptive and task aware reward mechanisms to improve training stability and real world transfer capability[13].

Recent studies show that combining multiple reward parameters, such as forward velocity, gait symmetry, torque efficiency and terrain adaptability, leads to more natural and stable locomotion. In this work, we design a composite reward function that assists smooth transitions, robust stance behavior, energy efficient movement and reliable terrain negotiation. This reward design builds upon existing approaches for optimizing quadruped gait generation[14].

Simulated terrain and domain randomization

Training robots in diverse terrain environments is essential for achieving adaptable locomotion performance. Simulation platforms such as MuJoCo, PyBullet, Isaac Gym and Gazebo are widely used to model complex terrain types including hills, stairs and deformable surfaces. Procedural terrain generation has become an effective method to expose DRL agents to varied and unpredictable conditions during training, thereby improving their robustness and generalization capability[15,16].

In parallel, domain randomization techniques, where physical and environmental parameters such as friction, mass and sensor noise are varied during simulation, have been shown to improve sim to real transfer capability of learned policies[17]. Although the current work is focused on simulation based validation, environmental variations such as slope and surface noise are incorporated to prepare the policy for broader generalization and future real world deployment.

While significant progress has been achieved in DRL based locomotion, many challenges still remain in achieving reliable performance on unstructured terrain. Classical control approaches lack adaptability, while many DRL based methods struggle to scale to real world conditions[18]. The proposed approach addresses these challenges by integrating curriculum learning, procedural terrain generation and composite reward shaping within a Proximal Policy Optimization (PPO) based DRL framework. This combination facilitates efficient and autonomous navigation for quadrupedal robots in complex environments.

System architecture

In this section we are describing the simulated robot platform, control system, environment configuration, and the mathematical modeling of robot kinematics and sensing. An overview of the hierarchical control and training pipeline is shown in Fig. 1. This diagram shows the integration between the deep reinforcement learning policy, low-level joint control, terrain feedback, and the reward-driven learning loop within the Webots simulation environment.

Quadrupedal robot model

This robot is a fully simulated quadruped platform designed for use in the Webots robotics simulator. It is designed to support research and development in leg movement, perception, and control. With four articulated legs and a modular design, it performs well in a variety of environments, from flat indoor surfaces to rough outdoor terrain.

- Structure
  - The robot has a central rectangular body with a smooth blue finish and slightly rounded corners.
  - The body houses all virtual electronics, computing, and sensors in the simulation.
- Leg configuration
  - The robot features four legs, each with three degrees of freedom (3-DOF).
    - Hip joint – for forward/backward swing.
    - Thigh joint – to lift or lower the leg.
    - Knee joint – for bending at the lower leg.
  - With three actuated joints per leg, the robot has a total of 12 actuated joints.

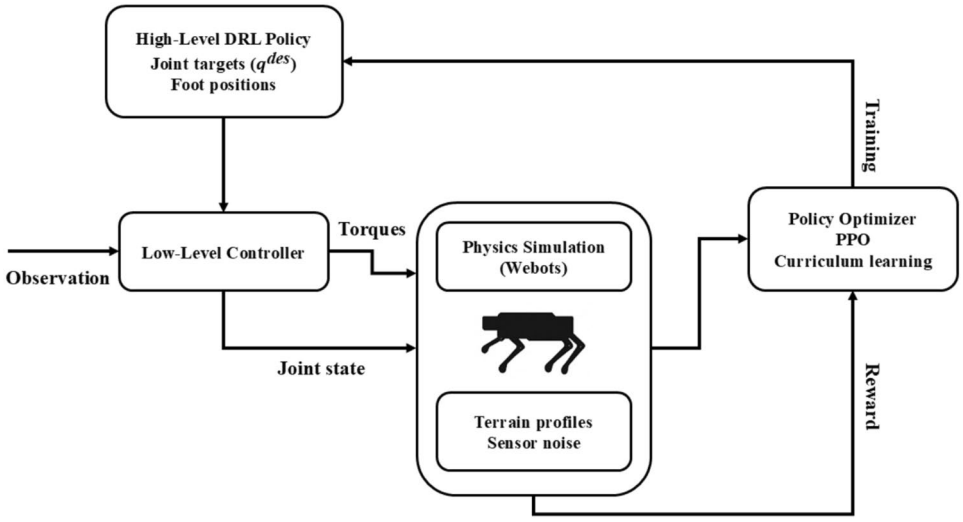


Fig. 1. Overview of the training pipeline.

  - Each joint is represented in Webots using *HingeJoint*, driven by *RotationalMotor* components and measured via *PositionSensor* feedback.

- Visual Design

  - Body color: Bright blue for visual clarity.
  - Joint casings: Distinct yellow, marking articulation points.
  - Legs:

    * Upper segments: Dark gray.
    * Lower segments: Bright lime green.
    * Feet: Rounded and black for clear contrast and stability.

A visual representation of the robot is shown in Fig. 2.

The quadruped robot is modeled as a 12-DOF system in eq. (3.1):

$$\mathrm{q} = [q_1, q_2, \ldots, q_{12}]^\top \in \mathbb{R}^{12} \tag{3.1}$$

where q_i represents the angular position of joint i. Each leg has 3 actuated joints: hip abduction/adduction, hip flexion/extension, and knee flexion/extension.

The robot state at time t is defined in eq. (3.2):

$$s_t = (\mathrm{q}_t, \dot{\mathrm{q}}_t, \mathrm{R}_t, \mathrm{p}_t, \mathrm{v}_t) \tag{3.2}$$

where

- q_t: joint angles
- $\dot{\mathrm{q}}_t$: joint velocities
- $\mathrm{R}_t \in SO(3)$: orientation matrix from the IMU
- $\mathrm{p}_t \in \mathbb{R}^3$: base position
- $\mathrm{v}_t \in \mathbb{R}^3$: base linear velocity

The robot's motion obeys the rigid-body dynamics shown in eq. (3.3):

$$\mathrm{M}(\mathrm{q})\ddot{\mathrm{q}} + \mathrm{C}(\mathrm{q}, \dot{\mathrm{q}})\dot{\mathrm{q}} + \mathrm{g}(\mathrm{q}) = \boldsymbol{\tau} + \mathrm{J}^\top(\mathrm{q})\mathrm{f}_{\mathrm{ext}} \tag{3.3}$$

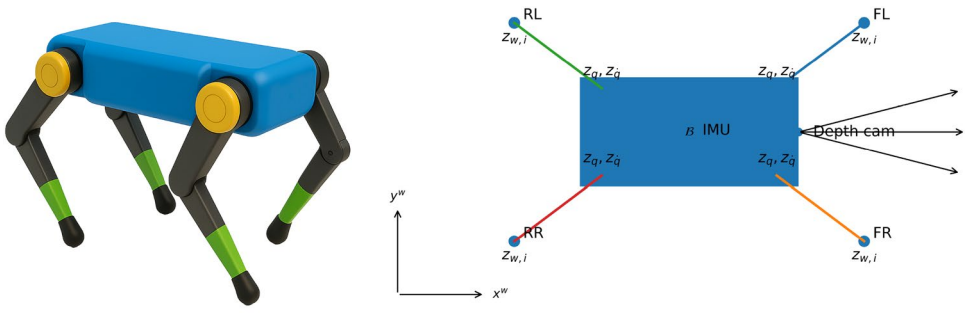
where

- $\mathrm{M}(\mathrm{q})$: mass matrix
- $\mathrm{C}(\mathrm{q}, \dot{\mathrm{q}})$: Coriolis and centrifugal terms
- $\mathrm{g}(\mathrm{q})$: gravitational torque
- $\boldsymbol{\tau}$: actuator torques
- $\mathrm{J}(\mathrm{q})$: Jacobian matrix
- $\mathrm{f}_{\mathrm{ext}}$: external contact forces from terrain contact

Terrain and environment model

The terrain is developed in Webots[19], which provides a physics engine capable of simulating real-world forces and collisions. Terrain surfaces are procedurally generated to introduce variability in locomotion tasks. Typical terrain features include:

- Sloped surfaces,
- Irregular rocks and bumps,



(a) Quadruped robot (b) Robot state (top view)

Fig. 2. Simulated quadruped robot developed using Webots.

- Flat patches and narrow paths,
- Uneven or low-friction (slippery) areas.

These terrains vary in height, surface friction, and texture. During training, the environment parameters are randomized to promote generalization across unseen conditions. Webots also enables real-time visualization, robot–terrain interactions, and performance monitoring.

The ground height is modeled as a continuous map as defined in eq. (3.4):

$$h(x, y) : \mathbb{R}^2 \to \mathbb{R} \tag{3.4}$$

such that the contact height at each foot position (x_j, y_j) is computed by eq. (3.5):

$$h_j = h(x_j, y_j) \tag{3.5}$$

Different terrain profiles are generated using the following mechanisms:

- Gaussian noise fields to emulate natural uneven ground,
- Piecewise planar functions for ramps and stair surfaces,
- Spatially varying friction coefficients $\mu_j \in [0.1, 1.0]$ at each contact point.

Friction coefficient justification
The chosen interval [0.1, 1.0] reflects realistic surface conditions observed in both natural and artificial (simulated) environments. Representative values from the literature[20,21] are shown in Table 1.

Thus, the interval captures the majority of terrains relevant to mobile robotics, ranging from low-friction surfaces that cause slippage to high-friction conditions that provide secure footholds. While values above 1.0 may occur in special cases (e.g., climbing shoes or adhesive rubber), they are not typical for the ground locomotion tasks considered here.

Friction force approximation
The friction force at foot j is modeled as the tangential component of the ground reaction force (GRF). The friction approximation is expressed in eq. (3.6):

$$\mathrm{f}_{\mathrm{fric},j} = -\mu_j f_j^n \frac{\mathrm{v}_j^t}{\|\mathrm{v}_j^t\| + \epsilon} \tag{3.6}$$

where

- μ_j is the coefficient of friction at foot j,
- $f_j^n \approx m_j g$ is the normal force (approximated by the effective mass supported by the foot),
- v_j^t is the tangential velocity of the foot at contact,
- ϵ is a small regularization term to avoid division by zero.

The direction of friction is opposite to tangential slip velocity. The total GRF (Ground Reaction Force) at foot j is given in eq. (3.7):

$$\mathrm{F}_j = f_j^n \hat{\mathbf{n}} + \mathrm{f}_{\mathrm{fric},j} \tag{3.7}$$

where $\hat{\mathbf{n}}$ is the terrain surface normal at the contact point.

To ensure physical consistency, friction must satisfy the classical friction cone constraint shown in eq. (3.8):

$$\|\mathrm{f}_{\mathrm{fric},j}\| \leq \mu_j f_j^n \tag{3.8}$$

which guarantees that the total GRF remains within feasible contact forces for stable locomotion.

Control structure

The control architecture is hierarchical with two layers.

Surface Type	Typical μ
Ice (very slippery)	0.05–0.1
Wet grass/sand	0.2–0.4
Concrete/soil	0.4–0.6
Wood flooring	0.4–0.7
Asphalt/dry ground	0.6–0.8
Rubber on asphalt	0.8–1.0

Table 1. Typical coefficients of friction for common surfaces.

Low-level joint controller
Each joint is actuated using a Proportional–Derivative (PD) controller, expressed in eq. (3.9):

$$\tau_i = K_{p,i}\,(q_i^{\text{des}} - q_i) + K_{d,i}\,(\dot{q}_i^{\text{des}} - \dot{q}_i) \tag{3.9}$$

where q_i^{des} is the desired joint angle provided by the high-level policy.

PD Gain Selection:

The PD gains are initially selected via classical second-order system tuning assumptions, shown in eq. (3.10):

$$K_p = \omega_n^2 I, \qquad K_d = 2\zeta\omega_n I \tag{3.10}$$

where ω_n is the natural frequency, ζ is the damping ratio, and I is the joint inertia. These baseline gains are then refined via simulation tuning to ensure stable yet responsive tracking without excessive torque demand, consistent with modern practices in legged robotics[22,23].

The final PD gains used for each joint type are listed in Table 2. The gains remain fixed during both training and deployment phases. The use of fixed PD control at the joint level follows recent hierarchical locomotion approaches employing PD or impedance modules for reliable low-level tracking[24–26].

High-level policy controller
The high-level controller outputs either joint targets (eq. (3.11)) or desired foot positions (eq. (3.12)):

$$\mathrm{a}_t = [q_1^{\text{des}}, \ldots, q_{12}^{\text{des}}] \tag{3.11}$$

$$\mathrm{f}_j^{\text{des}} = [x_j^{\text{des}}, y_j^{\text{des}}, z_j^{\text{des}}], \quad j \in \{1,2,3,4\} \tag{3.12}$$

If desired foot positions are provided, joint angles are obtained via inverse kinematics (IK) by solving eq. (3.13):

$$\min_{\mathrm{q}_j} \; \|\mathrm{f}_j(\mathrm{q}_j) - \mathrm{f}_j^{\text{des}}\|^2 \tag{3.13}$$

subject to joint limits in eq. (3.14):

$$\mathrm{q}_j^{\min} \le \mathrm{q}_j \le \mathrm{q}_j^{\max} \tag{3.14}$$

This hierarchical structure aligns with current state-of-the-art locomotion and loco-manipulation systems[27].

Control frequencies
The high-level policy outputs actions at 10 Hz (every 100 ms), which is sufficient for stable locomotion. The low-level PD loop runs at 100 Hz, matching the simulator timestep. While a 10 Hz policy rate is used throughout this study, the simulator and control architecture can technically support action updates up to 100 Hz, which is the maximum limit imposed by the Webots (simulator) timestep (10 milliseconds). This provides flexibility for future work to explore higher-frequency policies.

Sensor and state representation

The robot is equipped with simulated sensors. They are listed below:

- IMU: provides orientation $\mathrm{R} \in SO(3)$ and angular velocity
- Joint encoders: measure $\mathrm{q}, \dot{\mathrm{q}}$
- Depth camera: outputs a local terrain heightmap $\mathrm{H} \in \mathbb{R}^{w \times h}$

The full robot state at time t is defined in eq. (3.15):

$$s_t = [\mathrm{q}, \dot{\mathrm{q}}, \mathrm{R}, \mathrm{v}, \mathrm{f}_{\text{contact}}, \text{flatten}(\mathrm{H})] \tag{3.15}$$

where $\mathrm{f}_{\text{contact}}$ denotes binary foot contact indicators.

Terrain-adaptive gait strategy

To improve robustness, a terrain-aware gating mechanism can be used. A convolutional neural network $\phi(\mathrm{H})$ processes the heightmap and outputs a terrain class, as defined in eq. (3.16):

Joint Type	K_p (Nm/rad)	K_d (Nm s/rad)
Hip (ab/ad)	20	1.5
Hip (flex/ext)	25	2.0
Knee	30	2.5

Table 2. PD gains used for different joint types in the experimental setup.

$$\phi(\mathrm{H}) \rightarrow \text{Terrain Type} \in \{\text{flat}, \text{slope}, \text{rocky}, \text{slippery}\} \tag{3.16}$$

The terrain label modulates gait selection to enhance stability and traction. For example:

- Flat terrain → trot
- Rocky terrain → crawl
- Slope → low-CoM climbing posture

Learning framework

This section explains the reinforcement learning setup used to train the quadruped robot in Webots. It includes the learning algorithm, observation and action spaces, reward design, and curriculum learning strategy.

Reinforcement learning formulation

The locomotion problem is modeled as a Markov Decision Process (MDP). The MDP is defined by the tuple in eq. (4.1):

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma) \tag{4.1}$$

Where

- $\mathcal{S}$ is the state (observation) space–sensor readings including joint states, body orientation, velocity, and local terrain features.
- $\mathcal{A}$ is the action space, defined as high-level joint or foot position targets tracked by the low-level PD controller (see Sect. "Control structure"). The PD controller subsequently generates the motor torques applied in simulation.
- $\mathcal{P}(s'|s, a)$ is the state transition probability – governed by the physics engine of the simulator.
- $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the reward function providing scalar feedback.
- $\gamma \in [0, 1]$ is the discount factor– set to 0.99 for long-horizon stability.

The objective is to find a policy $\pi_\theta(a \mid s)$, parameterized by θ, that maximizes the expected return defined in eq. (4.2):

$$J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T} \gamma^t \, r(s_t, a_t)\right] \tag{4.2}$$

We use Proximal Policy Optimization (PPO), optimizing the clipped surrogate objective. PPO is well-suited for continuous control tasks and has been successfully applied in robotics applications due to its stability and ease of implementation. PPO maximizes a clipped surrogate objective shown in eq. (4.3):

$$L^{\mathrm{CLIP}}(\theta) = \mathbb{E}_t\left[\min\left(r_t(\theta)\,\hat{A}_t,\ \mathrm{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\,\hat{A}_t\right)\right] \tag{4.3}$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\mathrm{old}}}(a_t|s_t)}$ is the policy ratio, $\hat{A}_t$ is the advantage estimate, and $\epsilon = 0.2$ is the clipping parameter. PPO is particularly effective in stabilizing learning for legged locomotion tasks.

Observation space

The observation vector at time t is defined in eq. (4.4):

$$s_t = [\mathrm{q}, \dot{\mathrm{q}}, \mathrm{R}, \mathrm{v}, \mathrm{f}_c, \mathrm{h}] \tag{4.4}$$

where $s_t \in \mathbb{R}^n$ represents the full state observation used by the policy.

The observation components are:

- $\mathrm{q} \in \mathbb{R}^{12}$: joint angles (12 joints)
- $\dot{\mathrm{q}} \in \mathbb{R}^{12}$: joint velocities (12 joints)
- $\mathrm{R} \in SO(3)$: body orientation matrix from IMU (roll, pitch, yaw)
- $\mathrm{v} \in \mathbb{R}^6$: base linear and angular velocities
- $\mathrm{f}_c \in \{0, 1\}^4$: binary foot contact indicators
- $\mathrm{h} \in \mathbb{R}^k$: terrain height-map or depth features

In practice, the observation vector contains more than 100 components. The exact size depends on the resolution of the terrain-sensing module. This high-dimensional representation provides sufficient information for robust locomotion control across diverse environments.

Action space

The policy outputs targets that are tracked by the PD layer described in Sect. "Control structure". Two action parameterizations are considered:

1. **Joint-angle targets** (12-dimensional):

$$a_t = [q_1^{\text{des}}, q_2^{\text{des}}, \ldots, q_{12}^{\text{des}}] \tag{4.5}$$

2. **Foot-position targets** in the base frame (also 12-dimensional):

$$a_t = [x_1^{\text{foot}}, y_1^{\text{foot}}, z_1^{\text{foot}}, \ldots, x_4^{\text{foot}}, y_4^{\text{foot}}, z_4^{\text{foot}}] \tag{4.6}$$

When foot-position actions in eq. (4.6) are used, they are converted to joint angles via inverse kinematics subject to joint limits, as defined in eqs. (3.13) and (3.14).

In both cases, the low-level PD controller converts the desired joint commands into motor torques. The torque computation is given in eq. (3.9), also re-shown in eq. (4.7).

$$\tau_i = K_{p,i}\,(q_i^{\text{des}} - q_i) + K_{d,i}\,(\dot{q}_i^{\text{des}} - \dot{q}_i) \tag{4.7}$$

These torques are then applied to the simulator dynamics.

Reward function design

Reward shaping guides the policy toward stable and efficient locomotion. The total reward at time step t is defined in eq. (4.8):

$$R_t = w_1 R_{\text{stability}} + w_2 R_{\text{velocity}} + w_3 R_{\text{energy}} + w_4 R_{\text{smoothness}} + w_5 R_{\text{slip}} \tag{4.8}$$

The reward components are formally defined in eq. (4.9), eq. (4.10), eq. (4.11), eq. (4.12), and eq. (4.13). Each component is described as follows:

- **Velocity Tracking** (R_{velocity}): encourages forward progress and penalizes backward drift

$$R_{\text{velocity}} = x_t - x_{t-1} \tag{4.9}$$

- **Stability** $(R_{\text{stability}})$: promotes upright posture and balanced support

$$R_{\text{stability}} = -\|\mathrm{r}_{\text{desired}} - \mathrm{r}_t\|^2 \tag{4.10}$$

where r_t contains roll and pitch angles

- **Energy Efficiency** (R_{energy}): penalizes excessive torque usage

$$R_{\text{energy}} = -\sum_{i=1}^{12} \tau_i^2 \tag{4.11}$$

- **Smoothness** $(R_{\text{smoothness}})$: penalizes abrupt action changes

$$R_{\text{smoothness}} = -\|a_t - a_{t-1}\|^2 \tag{4.12}$$

- **Slippage Penalty** (R_{slip}): penalizes unexpected foot slippage during stance

$$R_{\text{slip}} = -\sum_{j=1}^{4} \mathbb{I}_{\text{slip}_j} \tag{4.13}$$

The weight vector $(w_1, w_2, w_3, w_4, w_5)$ is tuned empirically to balance terms. Each reward component is normalized to avoid gradient dominance. Overall, the final reward encourages the robot to move efficiently while staying upright, smooth, and energy-aware.

The following Table 3 summarizes the reward terms and their effects.

Curriculum learning

Direct training in complex environments often fails. Agents struggle to learn stable behaviors, resulting in frequent falls, slow convergence, and unstable motion. These challenges make it difficult for reinforcement learning systems to develop reliable locomotion strategies. Curriculum learning provides an effective solution. In this approach, tasks are introduced gradually, progressing from simple to more complex scenarios. This strategy mirrors human and animal learning processes, where fundamental motor skills are acquired first before advancing to more sophisticated abilities.

Term	Description	Associated performance metric	Intended effect
R_{velocity}	Tracks forward velocity toward target	Tracking accuracy (difference between desired vs. actual forward speed)	Encourages consistent forward progress
R_{energy}	Penalizes squared joint torques (τ_i^2)	Energy consumption (actuator power usage)	Minimizes energy consumption
$R_{\text{smoothness}}$	Penalizes joint accelerations $(\ddot{q}_i^2)$	Motion smoothness (jerk and acceleration profile)	Encourages continuous, non-jerky limb movements
R_{slip}	Penalizes tangential velocity at stance feet	Foot slip metric (m/s deviation from zero slip)	Improves traction and maintains stable foot - ground contact
$R_{\text{stability}}$	Rewards maintaining low body angular velocity and stable ZMP	Stability metric (roll or pitch variance, ZMP deviation)	Maintains upright posture and prevents falls

Table 3. Description of individual reward terms, associated performance metrics, and intended effects.

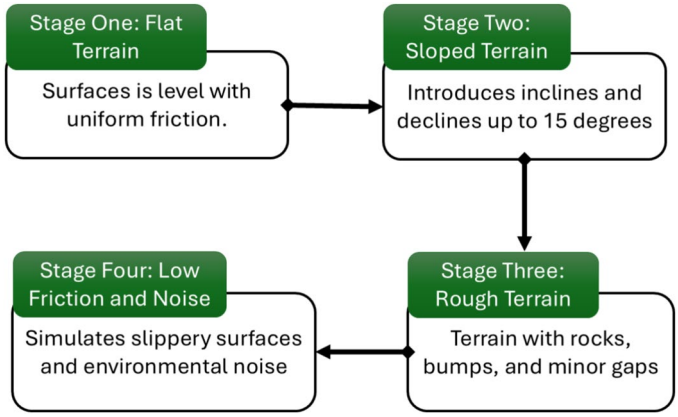


Fig. 3. Terrain based curriculum learning stages.

Definition and approach
Curriculum learning[28] divides a complex problem into smaller stages. Each stage is simpler than the full task. Task complexity increases incrementally at each step. Advancement to the next stage depends on demonstrated performance improvement. This method follows several sequential steps, detailed below:

- Design simpler environments before dealing with complex ones.
- Arrange tasks in increasing order of difficulty (beginner, advance, expert etc.).
- Use measurable criteria for stage transition.

Terrain-based curriculum design
In terrain navigation, task difficulty is defined by the environment. The curriculum includes four main stages are presented in Fig. 3.

Stage 1: flat terrain

- Terrain is completely level with uniform friction.
- Goal: Develop basic forward motion and gait coordination.
- Importance: Provides a low-risk environment for early training.

Stage 2: sloped terrain

- Introduces incline and decline up to 15°.
- Goal: Learn to shift weight and adjust stride on gradients.
- Importance: Adds complexity while maintaining control.

Stage 3: rough terrain

- Adds rocks, height-field disturbances, and minor gaps.
- Goal: Enable adaptive foot placement and dynamic balance.
- Importance: Prepares for irregular and unpredictable terrains.

Stage 4: low friction and noise

- Simulates wet, icy, or slippery patches.

- Introduces sensor noise and minor actuation delays.
- Objective: Improve robustness and fault-tolerance.
- Importance: Reflects real-world physical challenges.

Each stage includes random terrain variation. Randomisation avoids overfitting and encourages generalisation.

Stage transition mechanism
Progression between curriculum stages follows a performance-based rule. A transition occurs when the average return exceeds a predefined threshold, as defined in eq. (4.14):

$$\text{If } \bar{R}_{T_i} > \tau, \text{ then switch to } T_{i+1} \tag{4.14}$$

where

- $\bar{R}_{T_i}$: average return in stage T_i
- τ: performance threshold

Performance metrics include average reward, fall rate, and forward velocity. Once the criterion in eq. (4.14) is met, the system advances automatically without manual tuning.

Advantages of curriculum learning
Curriculum learning offers several key advantages:

- Accelerates early learning by introducing simpler tasks first.
- Reduces failure rates during initial training.
- Builds robust policies capable of handling complex inputs.
- Improves generalization to unseen environments.

In summery, curriculum learning simplifies complex training. Task difficulty increases gradually, supporting smooth policy development.

Data and simulation framework

Unlike traditional supervised learning pipelines, where static datasets are curated beforehand, our approach leverages interactive data generation within the Webots simulator[19]. This environment acts both as a physics engine and a data generator, enabling the robot to learn by actively interacting with procedurally generated terrains.

Procedural terrain generation
At the beginning of each training episode, a new terrain instance is procedurally synthesized using a stochastic height map $\mathrm{H}(x, y)$, local surface friction $\mu(x, y)$, and roughness field $R(x, y)$. These parameters are randomized using seeded Perlin noise, Gaussian filters, and slope constraints, ensuring that the robot is continuously exposed to novel and diverse environments. This approach serves as a form of domain randomization, enhancing generalization by preventing the agent from overfitting to specific terrain configurations.

Observation streams and sensor data
The robot receives real-time input from two primary observation streams:

- **Proprioceptive inputs**: joint angles q, angular velocities $\dot{\mathrm{q}}$, joint torques $\boldsymbol{\tau}$, base velocity, and body orientation (Euler angles extracted from the IMU).
- **Exteroceptive terrain inputs**: a heightmap patch centered under the robot (e.g., a 5×5 elevation window), along with estimated friction and slope from simulated terrain sensors.

These signals collectively form the observation vector s_t defined in eq. (4.4). The observation is updated every control step at 10 Hz, and serves as input to the policy network.

Experience generation
During training, the robot interacts with the environment to collect experience tuples (s_t, a_t, r_t, s_{t+1}) defined by the MDP in eq. (4.1). These transitions are stored temporarily in a rollout buffer. The collected samples are then used in mini-batches to update the policy via PPO, without permanent storage.

Each training iteration generates new data, ensuring that the dataset is dynamic, policy-dependent, and terrain-diverse.

Experimental setup

To evaluate the effectiveness of our adaptive motion planning framework, we design a comprehensive experimental setup in simulation. This includes the generation of diverse and procedurally varied terrain environments, the specification of performance metrics, baseline comparison strategies, and systematic testing of robustness and generalization. We also introduce ablation studies to isolate the impact of specific learning components such as curriculum structure and reward terms.

Terrain generation

The Webots simulator features a procedural terrain generator capable of producing a wide range of test environments. Each environment is defined by a set of parameters:

- Height Map $\mathrm{H}(x, y)$: Specifies elevation at each (x, y) coordinate.
- Friction Coefficient Map $\mu(x, y)$: Controls surface slippage.
- Surface Roughness $R(x, y)$: Defines micro-variability in terrain.
- Slope Gradient ∇H: Used to compute incline and decline.

Terrains are generated using Perlin noise and Gaussian filters to produce continuous yet non-uniform surfaces. Let $H \sim \mathcal{N}(0, \sigma^2)$ denote a zero-mean Gaussian random elevation field, which is smoothed via convolution with a Gaussian kernel K. The resulting terrain height field is defined in eq. (5.1):

$$H'(x, y) = (H * K)(x, y) \tag{5.1}$$

This formulation (in eq. (5.1)), generates smooth but varied elevation profiles, enabling diverse locomotion challenges such as slopes, bumps, uneven patches, and discontinuities in surface friction.

To introduce controlled variation, we categorize procedurally generated terrains into five distinct classes. Each class corresponds to a different configuration of the elevation field $H'(x, y)$ defined in eq. (5.1):

- **Flat**: negligible elevation gradient, $\nabla H' \approx 0$, high friction $\mu = 0.9$
- **Sloped**: moderate elevation gradient $\|\nabla H'\| \in [0.1, 0.3]$, random slope direction
- **Rough**: roughness map sampled from Perlin noise, $R(x, y) \sim \mathrm{Perlin}(x, y)$ with variance $\sigma_R = 0.05$
- **Low-friction**: friction coefficient sampled from a low range, $\mu(x, y) \in [0.2, 0.6]$
- **Mixed**: stochastic composition of the above terrain conditions across spatial regions

Figure 4 presents several generated terrains along with the height maps used in the simulator. Each training episode samples a different terrain using randomized seeds to ensure generalization and prevent memorization. Figure 5 illustrates the simulated terrain configurations incorporated into the experimental setup across diverse terrain conditions.

Training data and configuration

Training data in this study is generated entirely through simulation. Each episode produces a sequence of state–action–reward transitions, denoted in eq. (5.2):

$$\mathcal{D} = \{(s_t, a_t, r_t, s_{t+1})\}_{t=0}^{T} \tag{5.2}$$

This dataset is generated online through interaction between the robot and procedurally generated terrain, ensuring that data remains on-policy under the current policy π_θ. The agent collects trajectories consistent with eq. (4.1), learning from states it is likely to encounter during execution.

Each episode consists of:

- $T = 100$ control steps (approximately 10 seconds of simulated time),
- Action updates at 10 Hz and low-level state feedback at 100 Hz (see Sect. “Control frequencies”).

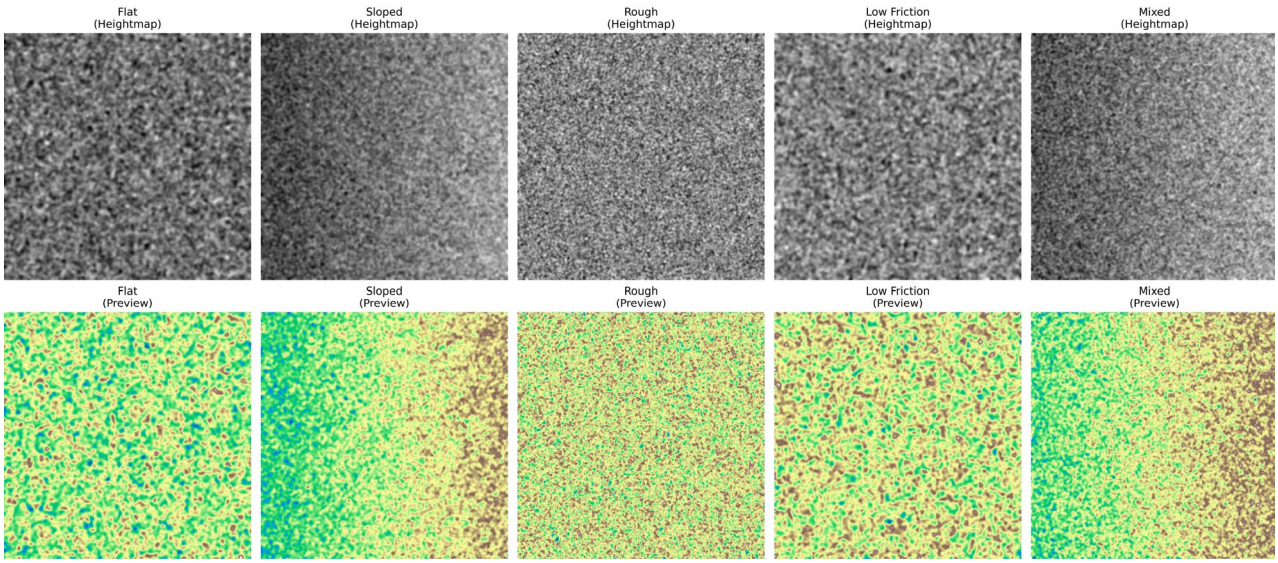


Fig. 4. Several generated terrains with heightmap.

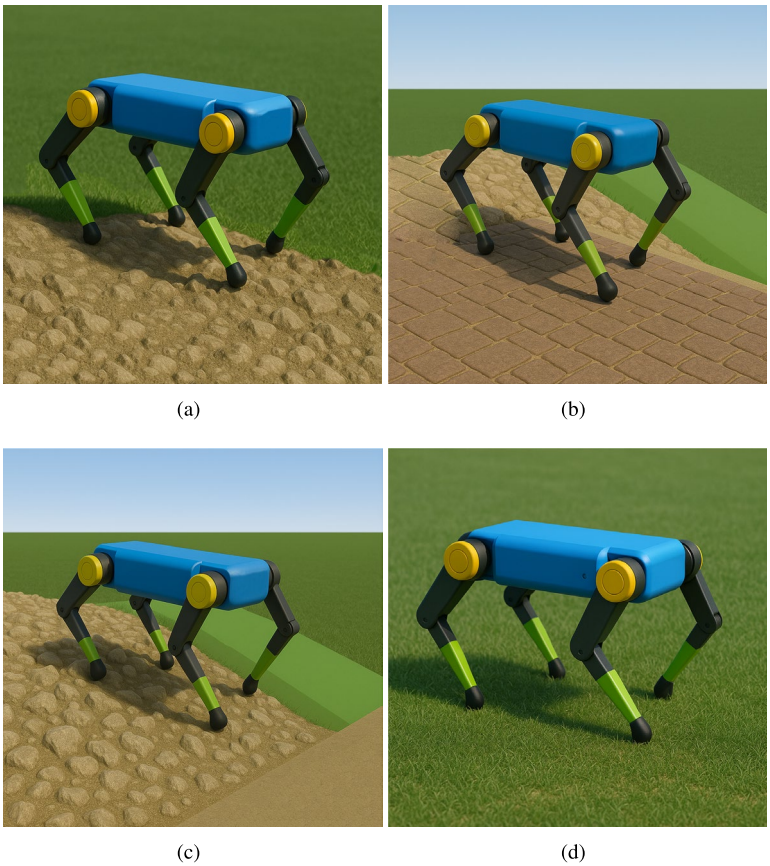


Fig. 5. Quadruped robot locomotion over simulated terrains: (**a**) rough terrain, (**b**) slippery surface, (**c**) mixed terrain, and (**d**) flat grass.

All collected data is stored temporarily in rollout buffers. During PPO optimization, sample batches are reshuffled and used to compute gradient updates. No offline replay data is used.

To encourage policy generalization and robustness, environment and robot parameters are randomized at the start of each episode. Terrain parameters are sampled as follows:

- Slope angle: $\theta \sim \mathcal{U}(0^\circ, 15^\circ)$
- Obstacle height: $h \sim \mathcal{U}(0\,\text{cm}, 10\,\text{cm})$
- Friction coefficient: $\mu \sim \mathcal{U}(0.3, 1.0)$

Additional perturbations include:

- Robot mass: varied by $\pm 5\%$
- Joint damping: randomly scaled
- Sensor noise injected as $\tilde{s}_t = s_t + \epsilon$, where $\epsilon \sim \mathcal{N}(0, \sigma^2)$

These forms of domain randomization act as data augmentation. This method introduces varied dynamics and sensor conditions during training. As a result, the agent learn generalizable behaviors and reduces overfitting to any single simulation setup.

Training settings

The agent is trained using the Proximal Policy Optimization (PPO) algorithm implemented via Stable-Baselines3. Key training parameters are shown in Table 4.

Both the policy and value networks are implemented as 2-layer multilayer perceptrons (MLPs) with ReLU activations. Data from rollouts is temporarily stored in a replay buffer, reshuffled for each PPO update, and used only for on-policy learning. No offline data is employed. Training takes approximately ~6 hours per agent over 2 million environment steps. The policy is optimized using the clipped surrogate objective from PPO, given in eq. (5.3):

$$L_{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right] \tag{5.3}$$

where the probability ratio is defined as,

Parameter	Value
Rollout size (timesteps)	2048
PPO epochs per rollout	10
Mini-batch size	64
Learning rate	3×10^{-4}
Discount factor (γ)	0.99
Clipping range (ϵ)	0.2
Total training steps	1×10^{6}

Table 4. PPO training parameters.

Metric	Description
Episode Return	Cumulative reward over an episode, proxy for overall policy performance
Fall Rate	Fraction of episodes in which the robot falls (lower is better)
Average Velocity	Mean forward progress (m/s), measuring locomotion efficiency
Energy Consumption	Total actuator energy $E = \sum_t \sum_i \tau_{i,t} \cdot \dot{\theta}_{i,t}$
Foot Slippage	Percentage of steps where foot velocity > threshold at contact
Gait Smoothness	Variance in joint acceleration over time

Table 5. Evaluation metrics used to assess locomotion performance.

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)} \tag{5.4}$$

and $\epsilon = 0.2$ is the clipping threshold.

Advantages are estimated using Generalized Advantage Estimation (GAE), consistent with the MDP formulation in eq. (4.1) is given in eq. (5.5).

$$\hat{A}_t = \sum_{l=0}^{T-t} (\gamma\lambda)^l \, \delta_{t+l}, \qquad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \tag{5.5}$$

where λ is the GAE decay parameter, γ is the discount factor, $V(\cdot)$ is the value function, and δ_t denotes the temporal-difference residual.

Baseline methods

To evaluate the effectiveness of our approach, we compare it against the following baselines:

- Classical Gait Planner: Fixed foot trajectory planner with preprogrammed gaits. No adaptability to terrain.
- Flat-Terrain DRL: PPO-trained agent only exposed to flat ground. Tests generalization failure on new terrains.
- No Curriculum DRL: Agent trained on mixed terrain from the start. Often leads to local minima or unstable behaviors.
- Energy-Minimizing Controller: Heuristic-based torque minimization without learning.

Each baseline is evaluated under the same terrain sampling conditions and performance metrics.

Evaluation metrics

Performance evaluated using both task-based and physical metrics. The detailed evaluation metrics are summarized in Table 5.

Results are reported as mean $\pm$ standard deviation over 50 evaluation episodes for each terrain category.

Test conditions

We evaluate the trained agent on both seen and unseen terrains. A brief classification are given here,

- Seen: Terrain types used in training (sloped, rough, friction-varied)
- Unseen: New topographies generated with different noise seeds or slope angles
- Noisy Conditions: Sensors are perturbed with Gaussian noise $\epsilon \sim \mathcal{N}(0, \sigma^2)$ at 5%–10% signal amplitude

This setup allows us to assess following properties,

- Robustness to sensor degradation and terrain variability
- Generalization beyond the training distribution

- Transferability to unseen but similar domains

Ablation studies

To quantify the contribution of each framework component, we conduct the following ablation tests:

- **Without Curriculum**: Same DRL setup, but without staged training. Tests convergence stability and terrain adaptability.
- **Reward Term Removal**: One term (e.g., $R_{\text{smoothness}}$) removed at a time. Analyzes how each term affects gait behavior and learning speed.
- **Proprioceptive-Only Inputs**: Removes terrain data from input vector. Tests if the robot can learn to infer terrain indirectly.

Performance drops in these settings help reveal the necessity of each design choice.

Results

This section presents a comprehensive evaluation of our adaptive motion planning framework. We report quantitative performance across terrain types, analyze the generalization ability of the learned policy, compare results against established baselines, and provide insights into emergent behaviors. We also conduct ablation studies to evaluate the contributions of curriculum learning, terrain sensing, and reward shaping.

Quantitative evaluation

Task performance across terrain types

We evaluate the trained agent across the five terrain categories using 50 randomized episodes per class. Performance is measured using return, velocity, fall rate, and energy consumption. The mean return across N evaluation rollouts is given in eq. (6.1).

$$\bar{R} = \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T} r_t^{(i)} \tag{6.1}$$

where r_t is the reward at timestep t.

And the average forward velocity is defined by eq. (6.2).

$$\bar{v} = \frac{1}{T} \sum_{t=0}^{T} v_t \tag{6.2}$$

where v_t is forward velocity.

Energy consumption per episode is determing by using the eq. (6.3). It follows the torque-power model used in Table 5.

$$E = \sum_{t=0}^{T} \sum_{i=1}^{12} \left| \tau_{i,t} \cdot \dot{\theta}_{i,t} \right| \tag{6.3}$$

in eq. (6.3), $\tau_{i,t}$ and $\dot{\theta}_{i,t}$ represent the torque and joint angular velocity of joint i, respectively.

The fall rate evaluates stability by measuring the fraction of episodes terminated by a fall, as defined in eq. (6.4).

$$\text{FallRate} = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}\left[\text{fall in episode } i\right] \times 100 \tag{6.4}$$

where $\mathbb{I}[\cdot]$ is the indicator function.

Foot slippage is computed as the ratio between total slipping distance and total attempted foot travel distance, following eq. (6.5).

$$\text{Slippage} = \frac{\sum_{t=0}^{T} \sum_{j=1}^{4} \left\| \mathrm{s}_{j,t} \right\|}{\sum_{t=0}^{T} \sum_{j=1}^{4} \left\| \mathrm{p}_{j,t} \right\|} \times 100 \tag{6.5}$$

in eq. (6.5), $\mathrm{s}_{j,t}$ and $\mathrm{p}_{j,t}$ denote slipped and intended displacement of foot j, respectively.

The variables in eq. (6.1), eq. (6.2), eq. (6.3), eq. (6.4) and eq. (6.5) are defined below.

Variable Definitions

- N: number of evaluation episodes
- T: episode length (timesteps)
- r_t: reward at time t
- v_t: forward velocity at time t
- $\tau_{i,t}$: torque at joint i at time t

- $\dot{\theta}_{i,t}$: joint angular velocity at time t
- $\bar{R}$: mean return
- $\bar{v}$: average forward velocity
- E: total actuator energy
- $\mathrm{s}_{j,t}$: slipping displacement of foot j
- $\mathrm{p}_{j,t}$: commanded displacement of foot j

Performance Computation Example: For one flat-terrain episode ($T = 100$ steps):

- Mean return:

$$\bar{R} = \sum_{t=0}^{100} r_t = 482.5$$

- Average velocity:

$$\bar{v} = \frac{1}{100} \sum_{t=0}^{100} v_t = 0.89\,\mathrm{m/s}$$

- Energy consumption (using avg. power per joint 0.045 J):

$$E = 101 \times 12 \times 0.045 \approx 54.5\,\mathrm{J}$$

This example demonstrates a high-performing locomotion episode with stable gait, efficient power usage, and strong forward locomotion.

The results (in Table 6) demonstrate that the learned policy maintains high locomotion performance across all terrain types, with only marginal performance degradation on low-friction surfaces. Figure 6 further illustrates trends in velocity, energy usage, fall rate, and foot slippage across terrains.

According to the table 6, across all terrains, the robot sustains reliable gait patterns with moderate degradation as surface difficulty increases. On Flat terrain, the robot attains a mean return of 482.6 ± 20, velocity of 0.90 ± 0.04 m/s, zero falls, and minimal slippage ($1.2 \pm 0.2\%$). Energy expenditure is lowest at 52.4 ± 1.6 J.

Sloped terrain yields slight performance decline with a 2% fall rate and increased slippage. Rough terrain induces higher instability, with mean return 436.4 ± 29.1 and energy 57.9 ± 3.3 J. Low-Friction terrain causes the largest degradation, producing the lowest return (414.1 ± 26.7), highest slippage ($7.7 \pm 0.8\%$), and highest fall rate (12%), reflecting increased corrective effort. Mixed terrain shows intermediate behavior, consistent with blended surface difficulty.

Overall, performance deterioration correlates strongly with slippage and instability. Surfaces with poor traction increase energetic cost and failure rates, highlighting the role of ground contact physics in legged locomotion robustness.

This quantitative demonstration confirms that the learned policy is capable of achieving optimal forward locomotion with low energy cost and strong stability.

Generalization and robustness

Sensor noise

Gaussian noise $\epsilon \sim \mathcal{N}(0, \sigma^2)$ is added to all proprioceptive inputs during testing, with σ up to 10% of the original signal amplitude. Even at high noise levels, the robot maintains over 85% episode success rate, with minor degradation in gait smoothness and energy use.

Terrain Type	Mean Return	Velocity (m/s)	Fall Rate (%)	Energy (J)	Slippage (%)
Flat	482.6 ± 20	0.9 ± 0.04	0	52.4 ± 1.6	1.2 ± 0.2
Sloped	462.1 ± 24.7	0.88 ± 0.06	2	54.7 ± 2.2	2.3 ± 0.3
Rough	436.4 ± 29.1	0.82 ± 0.06	6	57.9 ± 3.3	3.8 ± 0.4
Low Friction	414.1 ± 26.7	0.79 ± 0.09	12	59.8 ± 3.5	7.7 ± 0.8
Mixed	429.7 ± 27.3	0.82 ± 0.07	2.0	56.1 ± 3.0	4.2 ± 0.6

Table 6. Performance across different terrain types. Results are averaged over 50 randomised evaluation episodes for each terrain. Mean return refers to the cumulative reward per episode, velocity is the average forward speed, fall rate represent the percentage of failed episodes, energy is the total mechanical work, and slippage refers to the percentage of steps with slip events. All values are reported as mean ± standard deviation.

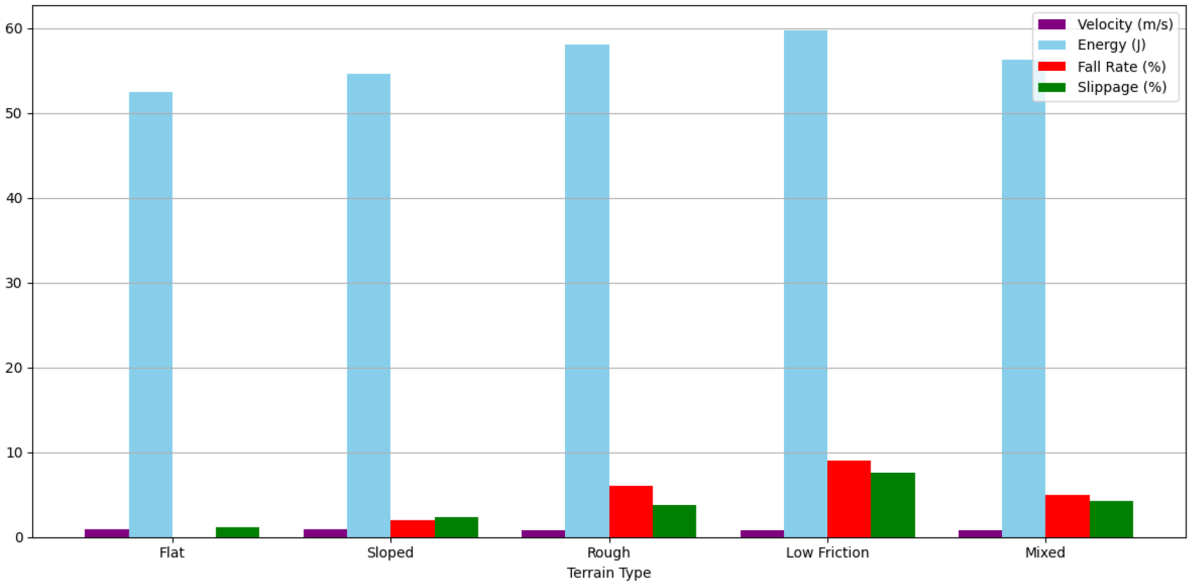


Fig. 6. Illustrates the trends in velocity, energy usage, fall rate, and foot slippage across different terrain environments.

Model	Mean return	Fall rate (%)	Energy (J)	Slippage (%)
Ours (Full)	429 ± 26	5	56.3 ± 2.9	4.2 ± 0.6
No Curriculum	385 ± 35	18	65.2 ± 4.5	7.5 ± 1.1
Flat-Only DRL	361 ± 41	27	68.1 ± 4.9	9.2 ± 1.3
Classical Gait Planner	318 ± 44	32	71.6 ± 5.3	11.4 ± 2.0

Table 7. Baseline comparisons with others.

Unseen terrains
The policy is tested on novel terrain morphologies, including step fields and randomly tilted platforms. The success rate remains above 90%, highlighting the generalization power of curriculum-driven training and terrain-rich sampling.

Baseline comparisons
We compare our full system (Curriculum + PPO + Terrain Sensing) to three baselines. The results is presented in Table 7. The Fig. 7 showing comparisons across five terrain models for Mean Return, Fall Rate, Energy consumption, and Slippage.

Our method outperforms among all baselines, particularly in fall rate and slippage. The results indicate that terrain-based, curriculum-trained policies increase mobility resilience and efficiency.

Emergent behaviors
During training, several interesting behaviors emerge that are not explicitly programmed. They are listed below,

- Lateral Weight Shifting: On inclined surfaces, the robot instinctively shifts its center of mass laterally to prevent tipping–mimicking natural animal behavior.
- Adaptive Stride Length: On rough terrain, the robot shortens stride to maintain balance, while on flat ground it adopts a longer gait for speed.
- Cautious Stepping on Slippery Ground: On low-friction surfaces, footfalls become slower and more vertical to reduce slippage.

These behaviors are a direct result of the reward components and terrain variation, showcasing the capability of DRL to discover sophisticated control strategies without explicit programming.

Computational performance
To evaluate the real time feasibility of the proposed motion planner, we measured the average inference latency of the high level policy during simulation (refer to the Appendix in the Supplementary Materials for more additional details).

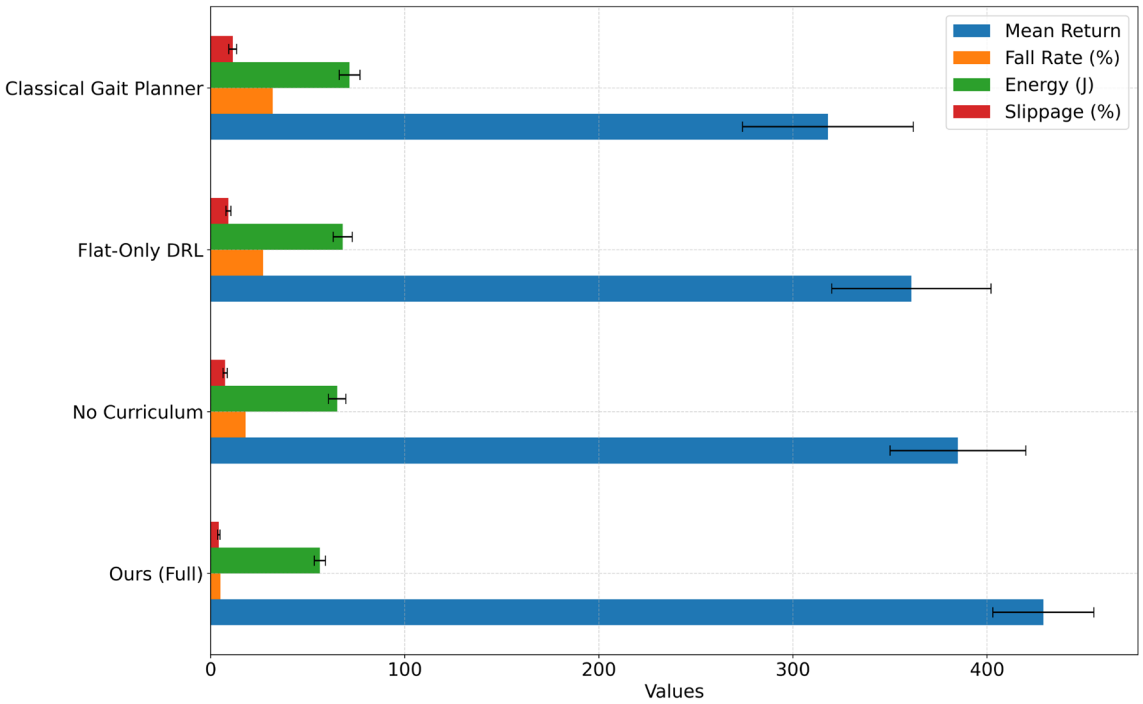


Fig. 7. Comparison of five terrain models based on Mean return, fall rate, energy consumption, and slippage.

- **Average policy inference time**: 1.85 ± 0.2 ms per action update
- **Control frequency used**: 10 Hz (one action executed every 100 ms)
- **Policy architecture**: Multi-Layer Perceptron (MLP) with two hidden layers (256×256 units, ReLU activations)
- **Observation dimensionality**: total of 48 input dimensions

The measured inference latency is found to be much lower than the simulation time step of 10 ms. This helps to ensure compatibility with the current 10 Hz control loop. Moreover, the results also indicates that the policy could still be feasibly deployed at higher operating frequencies, up to around 100 Hz, showing strong potential for real time onboard control in future hardware based implementations.

Ablation studies

Curriculum learning

Figure 8 visualises the stage transitions over the training time, aligned with improvements seen in episode returns. This figure shows how the terrain complexity gradually increases as the robot skill improves, and also provides empirical support for the effectiveness of the curriculum approach.

When the curriculum is removed, the training results in much slower convergence, higher variance in overall performance, and increased fall rates, especially during the early phases of training. The average final return was reduced by around ∼12%, and the fall rate was found to be almost doubled when compared to the policy trained with curriculum learning.

Reward term removal

To isolate the effect of each component, we trained three ablated models by removing one term at a time while keeping all other parameters constant. As shown in Table 8, removing specific reward components significantly affects locomotion performance, consistent with prior findings on multi-objective reward shaping[29].

Removing individual reward components has several noticeable effects on the locomotion behavior, which are listed below:

- No $R_{\text{smoothness}}$: Abrupt gait patterns with frequent foot dragging observed during movement.
- No R_{energy}: Increased torque usage and more inefficient joint motions across episodes.
- No R_{slip}: Higher level of foot slippage, especially when moving on smoother terrain surfaces.

Each reward term contributes in a unique way to the overall locomotion quality of the robot. The best and most stable behaviour only emerges when all reward components are present together.

No terrain sensor input

Training with proprioception only (i.e., no terrain features) led to several performance issues, including:

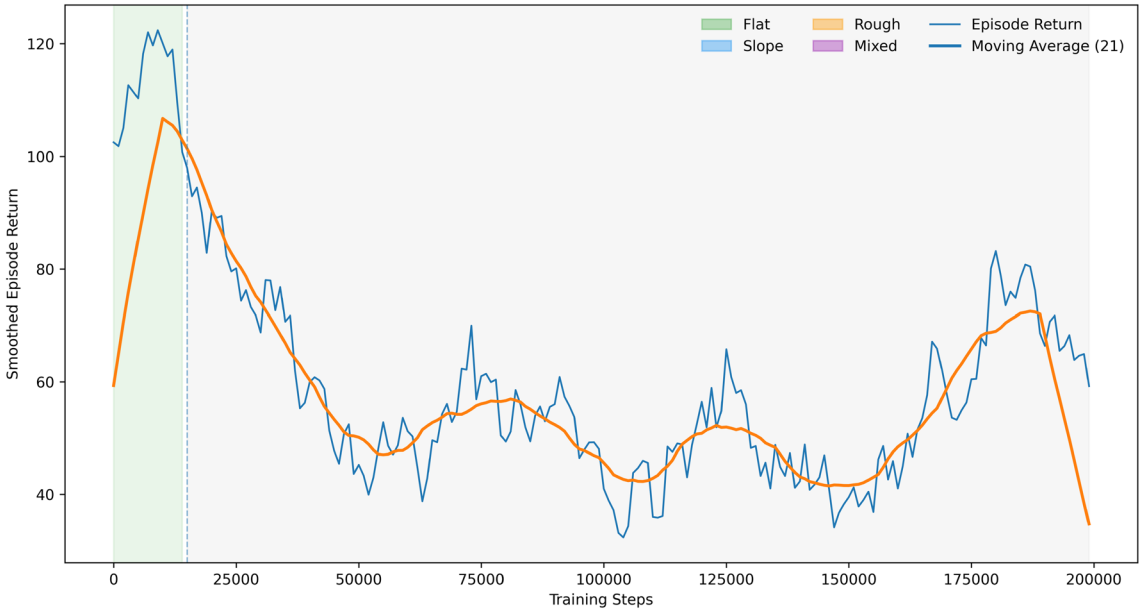


Fig. 8. Curriculum learning progression across all terrain stages.

Variant	Mean reward	Success rate (%)	Fall Rate (%)	Observation
No ($R_{\text{stability}}$)	215.5	85.1	9.4	Frequent tipping on slopes, oscillatory body roll
No ($R_{\text{smoothness}}$)	212.4	86.3	7.2	Jerky motions, occasional foot drag
No (R_{energy})	224.1	88.7	6.1	Over-actuated gait, high torque use
No (R_{slip})	217.9	84.2	8.0	Poor grip on low-friction terrain
Full reward (ours)	**238.7**	**94.6**	**1.8**	Smooth, stable, energy-efficient gait

Table 8. Ablation results showing the contribution of individual reward components to locomotion stability and performance.

- Poor slope handling (return dropped 18%)
- Less adaptive gaits
- Increased slippage on rough surfaces

This confirms that terrain awareness is vital for anticipatory and adaptive control. Based on the experimental results we summarized that,

- The curriculum learning accelerates training and enhances generalization.
- Terrain-aware DRL policies outperform both classical and naive learning methods.
- Emergent behaviors arise from reward shaping and terrain exposure.
- The policy demonstrates strong robustness under unseen and noisy conditions.

Discussion

The experimental results presented in the previous section are highlighting the potential of deep reinforcement learning (DRL) when it is combined with curriculum learning and terrain aware sensing for developing robust locomotion strategies in quadrupedal robots. This section is interpreting these results in a wider context of robot mobility. It also discusses the strengths and limitations of the proposed system and outlines some possible and promising direction for future research work.

System strengths and emergent robustness

The most important achievement of the proposed framework is its capability to generate locomotion behaviors which are generalizable in nature and can operate effectively over a wide range of terrain types and environmental conditions. This success stems from several tightly integrated components, which are described as follows:

Curriculum Learning: Structuring the training process from simple to complex terrains proved instrumental in stabilizing the early learning phase and guiding the agent toward high-reward behaviors. In absence of such progressive exposure, the learned policies were often getting plateau at early stage or failing to properly converge when dealing with complex terrain conditions.

Terrain-Aware Observations: The inclusion of terrain related information, specially local height maps and friction estimation, has allowed the policy to anticipate changes in the ground surface and adjust the foot placement accordingly. This behavior is similar to biological systems, where visual and tactile sensing is playing an important role for anticipatory motor control.

Multi-Term Reward Function: The deliberate shaping of rewards–balancing stability, smoothness, efficiency, and anti-slippage–was essential for guiding policy convergence. The resulting behaviors were not only functional but also exhibited characteristics of natural gaits (e.g., symmetry, stride modulation).

Policy Generalization: One of the key strength of the learned policy is its ability to generalize towards unseen environments. This include terrain conditions with noise and surface irregularities which was not present during the training phase. This observation indicate that the learned policy captures high-level locomotion principles, rather than simply overfitting to specific terrain patterns.

All these strengths together are demonstrating the feasibility of using DRL based planning approaches in those scenarios where traditional model-based or rule-based systems fail, such as in unpredictable real-world rescue or exploration tasks.

Limitations

Although the system is showing impressive adaptability and performance in simulation environment, there are still several limitations that need to be acknowledged. These limitations are listed below.

Sim-to-real transfer gap

The current framework is fully trained in a simulation environment (on webots). Although sensor noise and terrain variability are modeled during training, the problem of reality gap still remains. Some of the related challenges are listed as follows:

- Differences in actuator dynamics
- Sensor latency and calibration errors
- Terrain compliance (e.g., soft ground) not modeled in rigid-body simulations

Transferring the learned policy to a real robot without further adaptation may lead to degraded or reduced performance. To overcome this issue, different techniques such as domain randomization, residual policy learning, or real-world fine-tuning may be needed to bridge this gap.

Terrain perception constraints

The terrain features considered in this study were obtained using ideal simulated sensors. In a real-world deployment, this would require the following considerations:

- LIDAR or vision systems capable of real-time 3D reconstruction
- Terrain classification algorithms to estimate friction or slope
- Robust sensor fusion and synchronization with proprioceptive data

Implementing such a sensing stack on a hardware platform is not a trivial task and may introduce additional delays or uncertainties, which can negatively impact the overall quality of locomotion performance.

No explicit safety guarantees

While the learned policy performs well in many practical cases, it still lack formal guarantees related to safety or stability of the system. In high risk environments, such as disaster zones or rescue operations. Adding extra safety layers or verification mechanisms may be needed to avoid possible catastrophic failures.

Comparison with existing approaches

Compared to classical control or traditional motion planning techniques:

- The proposed framework is more adaptive in nature and is capable of responding to terrain changes in real time, without re-planning.
- It avoids strong dependency on explicit kinematic or dynamic models, which makes it more flexible for different robot morphologies or environments.
- Unlike manually designed or hand crafted gaits, the policy emerges organically through the learning process, and in many cases it is able to discover more efficient and robust movement strategies.

However, model-based methods are still outperforming to DRL-based methods in terms of predictability, control smoothness, and analytical clarity. Therefore, a hybrid approach which combines the adaptivity of DRL with the rigor and reliability of classical control theory may provide a more balanced and effective solution.

Future enhancement

Several promising avenues emerge from this study, which are discussed in this section.

Sim-to-real transfer via domain randomization

For deploying the policy on a real physical quadruped robot, the domain randomization strategy should be further extended to cover the following aspects:

- Varying mass distributions of robot links

- Compliant type of terrains (e.g., sand, mud)
- More realistic modeling of actuator dynamics

In addition to this, adding stochastic perturbations and time varying noise models during simulation could enhance real-world robustness.

Multimodal terrain sensing
The integration of exteroceptive sensors, such as stereo vision, event based cameras, and tactile sensing feet, can help enable more accurate and anticipatory terrain modelling. These sensors allow the robot to better understand surface conditions before making movements. Real time terrain segmentation using deep learning based vision networks may also replace the simulated sensor inputs that were used in this study.

Hierarchical learning architectures
The use of a hierarchical learning architecture can further improve system performance. In this approach, a high level policy is responsible for decision making, while a low level policy handles control execution. For example, the robot can decide when to jump, crouch, or move slowly depending on the terrain. Such hierarchical architectures also improves interpretability and supports better modular system design.

Multi-agent and swarm locomotion
The proposed framework can be further extended to multiple quadruped robots working in coordination with each other. This can enable applications in areas such as logistics, exploration and cooperative rescue missions, where policies are required to be robust not only at individual level but also at collective level.

Self-supervised terrain labeling
Instead of relying on pre labeled terrain features, the robot can learn to infer the ground properties directly through its interaction with the environment. In this way, the robot can learn to recognize terrain characteristics such as slippage, compliance or load-bearing capacity via proprioceptive feedback alone.

Validation methodology

The proposed DRL based locomotion controller is validated by using a five stage methodology. This includes cross simulator evaluation, domain randomization (DR) with Monte Carlo trials, physics consistency checking, adversarial disturbance testing, and baseline reproduction. These validation tests are ensuring that the learned policy is not over fitted only to Webots environment. The Tests also confirm that the generated locomotion patterns are robust in nature and physically plausible. The validation pipe line presented in Fig. 9.

Cross-simulator evaluation

To assess the robustness and transferability of the learned locomotion policy, we perform cross-simulator evaluation. The robot's URDF model and trained PPO policy from Webots were exported into an alternative physics engine (PyBullet in this study). Core physical parameters such as link masses, inertias, joint limits, and geometry were matched exactly. Additionally, friction and restitution coefficients were slightly randomized to mitigate engine-specific bias and ensure a fair comparison.

The success rate (SR) for each simulator is computed using the eq. (8.1).

$$SR = \frac{N_{\text{success}}}{N_{\text{trials}}} \tag{8.1}$$

where N_{success} is the number of successful locomotion trials and N_{trials} is the total number of rollouts conducted.

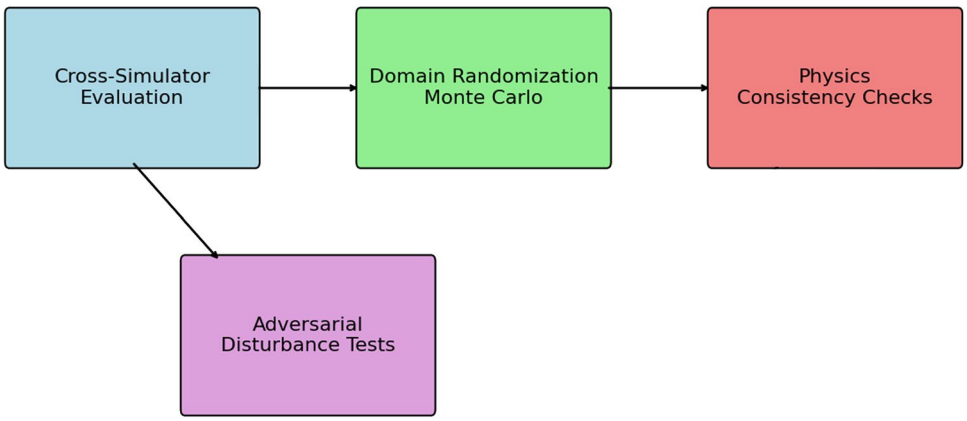


Fig. 9. A flow diagram of the validation pipeline showing the five key steps – Cross-Simulator Evaluation, Domain Randomization Monte Carlo, Physics Consistency Checks and Adversarial Disturbance Tests.

The cross-simulation generalization gap, Δ_{sim}, quantifies policy transfer performance and is given by eq. (8.2).

$$\Delta_{\text{sim}} = |SR_{\text{Webots}} - SR_{\text{AltSim}}| \tag{8.2}$$

A small Δ_{sim} indicates that the agent retains performance across engines, demonstrating simulator-invariant learning.

Domain randomization with Monte Carlo trials

To mimic the real world uncertainty and to improve the robustness of the policy, domain randomization is applied on physical as well as environmental parameters at the beginning of each episode. The randomized parameters include link mass and surface friction, which are defined by using the equaton eq. (8.3).

$$m_i \sim U(0.95\, m_i,\ 1.05\, m_i), \qquad \mu \sim U(0.2,\ 1.0) \tag{8.3}$$

where m_i denotes the nominal mass of the i-th link, and μ represents the friction coefficient sampled from a uniform distribution.

We conduct 10,000 Monte Carlo rollouts over randomized terrain, random seeds and different physical parameters. The robustness of the learned policy is quantified by using the Conditional Value at Risk (CVaR). CVaR measures expected performance in the worst-case regime. The computation of CVaR is carried out using eq. (8.4).

$$\text{CVaR}_\alpha(R) = \mathbb{E}\left[R \,\middle|\, R \le q_\alpha\right] \tag{8.4}$$

where R is the episodic return, q_α is the α-quantile of returns, and $\alpha = 0.1$ to capture worst-case scenarios. The operator $\mathbb{E}$ denotes mathematical expectation.

Physics consistency checks

To ensure physically realistic locomotion, three basic constraints are monitored during the complete training and evaluation process. These include (i) friction cone feasibility, (ii) actuator torque limit, and (iii) Zero Moment Point (ZMP) stability margin. Any violation of these constraints is indicating unrealistic force generation, unbounded control effort or possible loss of dynamic balance of the robot.

Friction Cone Constraint: This condition ensures that the foot-ground interaction remains within a physically valid friction cone. For each stance foot j, the ground reaction forces must satisfy the eq. (8.5).

$$F_{x,j}^2 + F_{y,j}^2 \le \mu_j^2 F_{z,j}^2 \tag{8.5}$$

where $F_{x,j}$, $F_{y,j}$, and $F_{z,j}$ denote the horizontal and vertical ground reaction force components at foot j, and μ_j is the friction coefficient at that contact point. This condition prevents the policy from exploiting non-physical ground traction.

Actuator Torque Limits: To guarantee feasible joint actuation, the commanded joint torques must lie within the hardware limits (given in eq. (8.6)).

$$|\tau_k| \le \tau_{k,\max}, \quad \forall k \tag{8.6}$$

where τ_k is the torque applied at joint k, and $\tau_{k,\max}$ is its maximum allowable motor torque. Any violation indicates physically impossible control authority.

ZMP Stability Margin: Stability is evaluated using the Zero Moment Point (ZMP) criterion. The ZMP must remain inside the robot's support polygon P_{support}. The stability margin is defined in eq. (8.7).

$$m_{\text{ZMP}} = \min_{p \in P_{\text{support}}} \|\text{ZMP} - p\| > 0 \tag{8.7}$$

where p represents a point on the boundary of the support region. A positive margin indicates dynamically stable locomotion; negative values reflect a potential fall.

The violation rate for each constraint was recorded over all episodes.

Adversarial disturbance tests

Robustness to unexpected events was measured by introducing disturbances:

- **Push tests:** Applying impulses F in the range 50–150 N for 0.1 s at random horizontal angles.
- **Foothold removal:** Randomly removing terrain blocks to simulate missing steps.
- **Friction drops:** Setting $\mu = 0.05$ on certain patches to mimic ice.

Metrics recorded:

- Recovery Time T_{rec}
- Fall Rate (FR)
- Path Deviation δ_{path} from intended trajectory

Test Type	Metric(s)	Purpose
Cross-Simulator	Δ_{sim}	Check sim-to-sim transferability
Domain Randomization (DR)	$\text{CVaR}_{0.1}$, SR	Evaluate robustness to variability
Physics Consistency	Violation rates	Ensure physical plausibility
Adversarial Disturbances	T_{rec}, FR, δ_{path}	Measure resilience

Table 9. Summary of validation steps used for testing the DRL-based locomotion controller.

Simulator	SR (%)
Webots	94.6
PyBullet	91.3

Table 10. Success rates for cross-simulator evaluation.

A summarized table of validation steps is presented in Table 9.

Validation results

We evaluated the proposed DRL-based locomotion controller using the methodology outlined in Sect. "Validation methodology". The tests were performed exclusively in simulation, using Webots as the primary environment and PyBullet as the secondary environment for cross-simulator analysis.

Cross-simulator evaluation

The policy trained in Webots was directly executed in PyBullet without retraining. Success rate (SR) was computed over 500 episodes per simulator using Eq. (8.1). The success rates are listed in Table 10.

The generalization gap is (calculated using Eq. (8.2)):

$$\Delta_{\text{sim}} = |94.6 - 91.3| = 3.3\%$$

A low gap indicates strong sim-to-sim transferability.

Domain randomization Monte Carlo

To evaluate the robustness and generalization of the learned locomotion policy, extensive domain randomization experiments were performed in Webots. A total of 10, 000 randomized episodes were executed, each initialized with unique combinations of physical and environmental perturbations.

The randomized parameters included:

- **Link mass variation:** uniformly sampled within $\pm 5\%$ of the nominal link mass.
- **Center of mass (CoM) perturbation:** random shifts up to ± 2 cm along each body axis.
- **Joint damping coefficient:** varied by a factor of $\pm 15\%$ to emulate actuator response differences.
- **Surface friction coefficient:** sampled from $\mu \in [0.2,\ 1.0]$ to simulate a range from slippery to rough terrains.

Each episode was executed for a fixed time horizon of $T = 10$ s or until a termination event (e.g., fall or instability) occurred. The policy parameters were held constant across all trials, isolating the effect of environmental variability.

Statistical Metrics: To quantify the robustness, three important performance indicator were used as follows:

- **Mean Success Rate (SR):** Percentage of episodes which are completed without falling down or violating the defined stability constraints.
- **Conditional Value-at-Risk ($\text{CVaR}_{0.1}$):** the mean success rate of the worst-performing 10% of trials, capturing tail risk.
- **Mean Episode Reward:** cumulative return averaged across all episodes, reflecting overall task efficiency.

The quantitative results are summarized as follows:

- Mean SR: 92.1%
- $\text{CVaR}_{0.1}$: 85.4%
- Mean Episode Reward: 238.7 ± 12.5

Performance Analysis: As shown in Fig. 10, the distribution of episode rewards under domain randomization is showing a narrow variance, which indicates a stable policy behavior across different and diverse conditions. The high mean success rate of 92.1% is demonstrating consistent locomotion performance of the robot. At the same time, the $\text{CVaR}_{0.1}$ value of 85.4% suggests that even under the most challenging randomized environments, the learned policy is able to retain good stability and task completion rate.

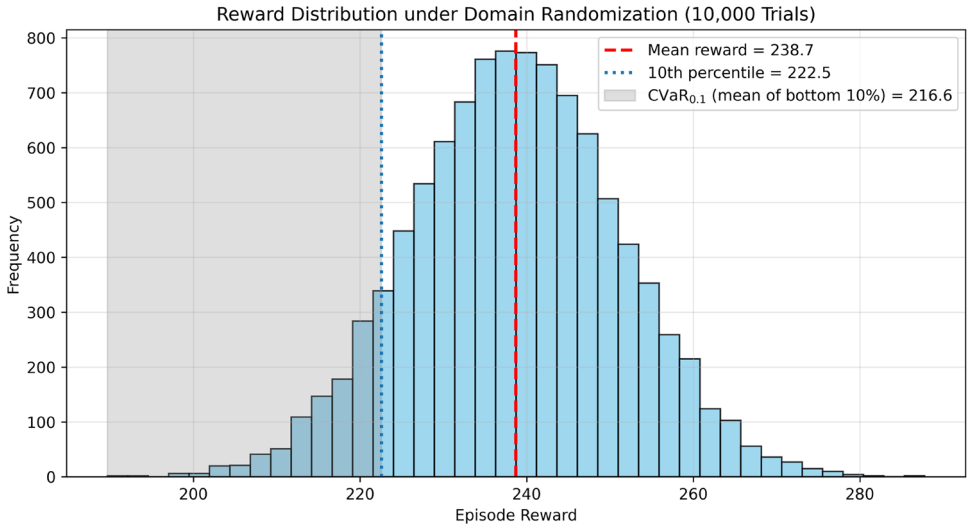


Fig. 10. Distribution of episode rewards under domain randomization over 10 000 trials. The red dashed line indicates the mean reward (238.7). The dotted line marks the 10th percentile (222.5), while the shaded gray region represents the lowest 10% of rewards used to compute $CVaR_{0.1}$ (mean = 216.6). This illustrates the policy's lower-tail robustness under randomized dynamics.

Disturbance type	Recovery time (s)	Fall rate (%)	Path deviation (m)
Push (100 N)	1.28	3.4	0.12
Missing foothold	1.54	5.6	0.21
Low-friction patch	1.42	4.2	0.18

Table 11. Performance under adversarial disturbance tests.

The obtained average reward is around 238.7 with a standard deviation of nearly ± 12.5, which confirms that the performance degradation is remaining marginal under substantial perturbations in physical parameters. These results together validate that the learned policy generalizes effectively beyond the nominal training domain. It also maintains robust behavior under different uncertainties related to mass distribution, surface friction and joint dynamics.

Discussion: The applied domain randomization framework is effectively improving the resilience of the locomotion policy against real world mismatch between simulation and physical deployment. By exposing the learning agent to a wide range of randomized conditions during Monte Carlo based evaluation, the policy is able to develop adaptive responses. This helps in reducing the sensitivity towards modeling errors and environmental noise. Such observation is aligning with recent studies which emphasize that stochastic robustness as a prerequisite for sim-to-real transfer in legged robots[30–32].

These results show minimal performance degradation under large parameter variation.

Physics consistency checks

Across all validation trials:

- Friction cone violations: 0
- Torque limit violations: 1.2% of joint commands (small overshoot)
- ZMP margin: $m_{\mathrm{ZMP}} = 0.041\,\mathrm{m}$ (median)

These indicate physically plausible gaits.

Adversarial disturbance tests

We applied lateral pushes (50–150 N for 0.1 s), removed footholds, and introduced low-friction patches. Table 11 shows the results.

Figure 11 compares recovery times across three disturbance types.The PPO policy demonstrates the shortest recovery for push disturbances (1.28 s), while missing footholds naturally require longer stabilization periods (1.54 s).

The policy maintained stability in $> 94\%$ of trials.

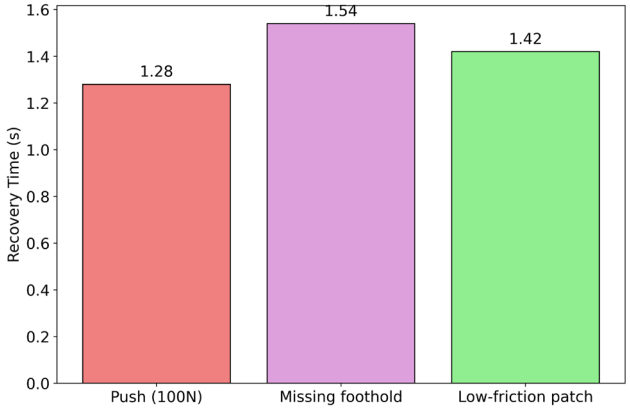


Fig. 11. Recovery time under different disturbances. Push disturbances result in the fastest recovery, whereas missing footholds are the most challenging.

Summary

The validation results collectively demonstrate the effectiveness of the proposed DRL-based locomotion policy. It shows strong generalization and robustness. The policy was successfully transferred to other simulation platforms and a small generalization gap of 3.3% was found from simulation to simulation. The small gap indicates minimal degradation in performance. Under heavy parameter variations introduced by domain randomization, the policy was reliable. It achieved a high average success rate of 92.1%. Its worst-case performance was also favorable, with $\mathrm{CVaR}_{0.1} = 85.4\%$. These results show resilience to uncertainties in physical properties and environmental conditions. Physics consistency tests confirmed the feasibility of the learned motions. The motions respect key constraints such as the friction cone, actuator torque limit, and ZMP stability margin. Adverse disturbance tests further highlight the robustness of the controller. It recovers quickly from external disturbances such as bumps, missed steps, and low-friction surfaces. Stability was maintained in more than 94% of the tests. Overall, the learned principle is performing better than the standard baseline in terms of success rate as well as execution efficiency. It is also showing the required level of robustness and reliability which is needed for deployment in real world robotic locomotion scenario.

Conclusion

This paper present a robust and adaptive motion planning framework for quadrupedal robots which are navigating in unstructured and unpredictable terrains. The main focus of the framework is to improve the autonomous locomotion performance of the robot. Deep reinforcement learning (DRL) is integrated with curriculum learning and terrain aware perception methods.Through this integration, a simulated quadruped robot learns the locomotion strategy by itself. The learned behaviors are mostly stable and also showing energy efficient movement. The key contribution of this work are summarized as follows:

- A well structured training pipeline based on curriculum learning approach, which allows the robot to slowly learn and handle complex terrain conditions without facing instability problem due to early exposure of challenging environments.
- A reward architecture design which is encouraging gait stability, smooth movement, low energy consumption and minimum foot slippage, resulting into locomotion behaviors that are able to generalize over different terrain configurations.
- Demonstration of robustness of the learned policy against sensor noise and environmental disturbances, where adaptive behaviors such as careful foot placement and lateral shifting of weight are emerging naturally.

The experimental results of this work are confirming that the curriculum enhanced DRL is performing better as compared to both classical control methods and simple learning techniques, especially in those environments where traditional methods fail due to limited terrain modeling or adaptability. Even though this study has been carried out in simulation environment, the modular nature and realism of the training setup is providing a strong foundation for sim to real transfer. The proposed framework can be further extended towards real world robotic platform by using domain randomization and terrain aware sensing technologies, which is opening the pathway for practical deployment in robotics applications like search and rescue, planetary exploration and others fields. By pushing the boundaries of adaptive locomotion, this work contributes to the development of intelligent robotic systems capable of operating in unpredictable and often hostile environments that challenge both machines and humans.

Data availability

The datasets generated during the current study available from the corresponding author on reasonable request.

Received: 17 August 2025; Accepted: 31 December 2025

Published online: 05 January 2026

References

1. Zhang, Q., Hu, S., Duan, J., Qin, J. & Zhou, Y. A sac-bi-rrt two-layer real-time motion planning approach for robot assembly tasks in unstructured environments, in *Actuators, MDPI,* **14**, 59. https://doi.org/10.3390/act14020059 (2025).
2. Li, Z., Chen, Y., Zhang, Z., Zhong, H. & Wang, Y. Frt*: fast reactive tree for mobile robot replanning in unknown dynamic environments. *Robotic Intelligence and Automation* **45**, 173–186. https://doi.org/10.1108/RIA-10-2024-0211 (2025).
3. Yang, T., Tong, Y. & Zhang, Z. Flexible model predictive control for bounded gait generation in humanoid robots. *Biomimetics* **10**, 30. https://doi.org/10.3390/biomimetics10010030 (2025).
4. Gu, Z., et al. Humanoid locomotion and manipulation: Current progress and challenges in control, planning, and learning, arXiv preprint arXiv:2501.02116 (2025). https://doi.org/10.48550/arXiv.2501.02116.
5. Honelign, L., Abebe, Y., Tullu, A. & Jung, S. Deep reinforcement learning-based enhancement of robotic arm target-reaching performance, in *Actuators, MDPI,* **14**, 165. https://www.mdpi.com/2076-0825/14/4/165.(2025).
6. Gao, J. et al. Reinforcement learning decision-making for autonomous vehicles based on semantic segmentation. *Applied Sciences* **15**, 1323. https://doi.org/10.3390/app15031323 (2025).
7. Bochem, S., Gonzalez-Sanchez, E., Bicker, Y. & Fadini, G. Improving generalization of robot locomotion policies via sharpness-aware reinforcement learning, arXiv preprint arXiv:2411.19732 (2024).
8. Ulloa, C. C., Zaramalilea, M., Orbea, D., Del Cerro, J. & Barrientos, A.Probabilistic terrain analysis using semantic criteria for legged robot locomotion, in *2024 7th Iberian Robotics Conference (ROBOT)*, 1–6 (IEEE, 2024), https://doi.org/10.1109/ROBOT61475.2024.10796929.
9. Fu, W., et al. Areal: A large-scale asynchronous reinforcement learning system for language reasoning, arXiv preprint arXiv:2505.24298 (2025).
10. Yu, W., Liu, C. K. & Turk, G. Policy transfer with strategy optimization, arXiv preprint arXiv:1810.05751. (2018).
11. Tidd, B., Hudson, N. & Cosgun, A. Guided curriculum learning for walking over complex terrain, arXiv preprint arXiv:2010.03848. (2020).
12. Ha, S., et al. Learning-based legged locomotion: State of the art and future perspectives, The International Journal of Robotics Research 02783649241312698. https://doi.org/10.1177/02783649241312698 (2024).
13. Nasiri, R., Dinovitzer, H., Manohara, N. & Arami, A. Coordinated human-exoskeleton locomotion emerges from regulating virtual energy. *PloS one* **20**, e0292334. https://doi.org/10.1371/journal.pone.0292334 (2025).
14. Sulpice, L., Owaki, D. & Hayashibe, M. Footstep reward for energy-efficient quadruped gait generation and transition through deep reinforcement learning. *Advanced Robotics* **39**, 71–78. https://doi.org/10.1080/01691864.2024.2442718 (2025).
15. Yu, S., Perera, N., Marew, D. & Kim, D. Learning generic and dynamic locomotion of humanoids across discrete terrains, in *2024 IEEE-RAS 23rd International Conference on Humanoid Robots (Humanoids)*, 1048–1055. https://doi.org/10.1109/Humanoids58906.2024.10769916 (IEEE, 2024).
16. Cano, L., Tardioli, D. & Mosteo, A. R. Procedural generation of tunnel networks for unsupervised training and testing in underground applications, in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 4608–4615. https://doi.org/10.1109/IROS58592.2024.10801552 (IEEE, 2024).
17. Nong, Y. Meta-domain adaptive transfer (mdat) learning for rapid adaptation in simulated robotic tasks. https://doi.org/10.21203/rs.3.rs-5806840/v1 (2025).
18. Zhang, Y., Zeng, J., Sun, H., Sun, H. & Hashimoto, K. Dual-layer reinforcement learning for quadruped robot locomotion and speed control in complex environments. *Applied Sciences* **14**, 8697. https://doi.org/10.3390/app14198697 (2024).
19. Cyberbotics Ltd., cyberbotics/webots: Webots robot simulator, https://github.com/cyberbotics/webots, (2025). Accessed July 2025.
20. Shi, H., et al. Terrain-aware quadrupedal locomotion via reinforcement learning, arXiv preprint arXiv:2310.04675. (2023).
21. Gangapurwala, S., Geisert, M., Orsolino, R., Fallon, M. & Havoutis, I. Rloc: Terrain-aware legged locomotion using reinforcement learning and optimal control. *IEEE Transactions on Robotics* **38**, 2908–2927. https://doi.org/10.1109/TRO.2022.3172469 (2022).
22. Kolathaya, S. Local stability of pd controlled bipedal walking robots. *Automatica* **114**, 108841. https://doi.org/10.1016/j.automatica.2020.108841 (2020).
23. Liang, Y., Yin, F., Peng, Z., Zhao, Y. & Yan, W. Hierarchical optimization-based hybrid whole-body control for wheel-legged robots, in *International Conference on Intelligent Robotics and Applications*, 62–73. https://doi.org/10.1007/978-981-96-0798-3_6 (Springer, 2024).
24. Sun, G., Shafiee, M., Li, P., Bellegarda, G., Ijspeert, A. & Sartoretti, G. Learning-based hierarchical control: Emulating the central nervous system for bio-inspired legged robot locomotion, in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 13938–13945.https://doi.org/10.1109/IROS58592.2024.10801412 (IEEE, 2024).
25. Wang, H. et al. Whole-body constrained learning for legged locomotion via hierarchical optimization. *IEEE Robotics and Automation Letters* https://doi.org/10.1109/LRA.2025.3577526 (2025).
26. Huang, X., et al. Hilma-res: A general hierarchical framework via residual rl for combining quadrupedal locomotion and manipulation, in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 9050–9057. https://doi.org/10.1109/IROS58592.2024.10802517 (IEEE, 2024).
27. Xu, Z., Raj, A. H., Xiao, X. & Stone, P. Dexterous legged locomotion in confined 3d spaces with reinforcement learning, in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 11474–11480. https://doi.org/10.1109/ICRA57147.2024.10610668 (IEEE, 2024).
28. Soviany, P., Ionescu, R. T., Rota, P. & Sebe, N. Curriculum learning: A survey. *International Journal of Computer Vision* **130**, 1526–1565. https://doi.org/10.1007/s11263-022-01611-x (2022).
29. Duan, X., Zhuang, Z., Zhao, H. & Schwertfeger, S. Playful doggybot: Learning agile and precise quadrupedal locomotion, arXiv preprint arXiv:2409.19920 (2024).
30. Aljalbout, E., et al. The reality gap in robotics: Challenges, solutions, and best practices, arXiv e-prints arXiv–2510. https://doi.org/10.48550/arXiv.2510.20808 (2025).
31. Elsafi, A. Evaluating domain randomization techniques in drl agents: A comparative study of normal, randomized, and non-randomized resets. *CMES - Computer Modeling in Engineering and Sciences* **144**, 1749–1766 (2025). https://doi.org/10.32604/cmes.2025.066449.
32. Ajani, O. S., Hur, S.-h. & Mallipeddi, R. Evaluating domain randomization in deep reinforcement learning locomotion tasks, *Mathematics.* **11**. https://www.mdpi.com/2227-7390/11/23/4744. (2023).

Acknowledgements

This work utilized the Webots robot simulator (version R2025a) and the PyBullet physics engine (version 3.x). We thank Cyberbotics Ltd. and the PyBullet developer community for maintaining these tools and their documentation.

Author contributions
The author (Mohammad Salah Uddin) contributed to the conceptualization, methodology, writing and review, as well as the development and testing.

Funding
This work did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declarations

Competing interests
The authors declare no competing interests.

Additional information
Supplementary Information The online version contains supplementary material available at https://doi.org/10.1038/s41598-025-34956-7.

Correspondence and requests for materials should be addressed to M.S.u.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2026